

Parte 1: Fundações e Arquitetura

Contexto: Integração de Tecnologias Web e Inteligência Artificial

Foco Pedagógico: Arquitetura de Software, Spec-Driven Development, Clean Code e Isolamento de Domínio

1. Visão Geral e Paradigma de Desenvolvimento

Nesta primeira etapa, o foco não é a digitação manual de linhas de código, mas sim a concepção arquitetural da "base da casa". Vocês integrarão **React (front-end)** e **Spring Boot (back-end)** em um ecossistema coeso, projetado para escalar e receber integrações futuras de IA.

Mais importante do que *o que* vamos construir é *como* vamos construir. O desenvolvimento moderno exige que a engenharia de software seja guiada por especificações rigorosas e ferramentas assistivas, elevando o papel do desenvolvedor a um arquiteto de soluções.

2. Princípios Metodológicos Norteadores

O fluxo de trabalho desta sprint abandona a codificação manual direta em favor de um processo orientado a especificações (design-first). Vocês deverão seguir estritamente os dois princípios operacionais abaixo:

- **Spec-Driven Development (SDD) com Opencode:** O código final é um subproduto de uma especificação bem elaborada. Vocês utilizarão o *opencode* como motor de criação para conduzir este processo. **Tarefa preparatória:** É essencial que pesquisem e compreendam a fundo o conceito de *Spec-Driven Development*. Entendam como a definição clara do comportamento, das interfaces e dos contratos do sistema deve preceder qualquer implementação técnica.
- **Engenharia de Prompts (Padrão CRISP):** A comunicação com as ferramentas de IA para geração de software requer precisão. Todos os prompts elaborados pela equipe devem manter e respeitar o padrão **CRISP**. O fluxo de trabalho obrigatório consiste em duas etapas: primeiro, utilizar prompts CRISP para gerar os documentos de especificação do sistema (System Docs) detalhados; em seguida, usar esses documentos validados como o contexto base para a geração automatizada dos códigos estruturais.

3. Arquitetura do Back-end (Spring Boot)

Objetivo Conceitual: Projetar uma API REST que atue como a orquestradora central do domínio, garantindo uma separação absoluta de responsabilidades.

A arquitetura deve refletir um isolamento de domínio rigoroso. O sistema deverá expor contratos claros para o fluxo de conversas, recepção de anexos e monitoramento de saúde */health*. A regra fundamental de design nesta camada é a blindagem do núcleo da aplicação:

- **Controladores são apenas fronteiras HTTP:** Eles atuam como tradutores, recebendo requisições da web e devolvendo payloads de resposta, mas jamais devem conter lógicas de negócio ou conhecimento profundo das regras do sistema.
- **Serviços detêm o monopólio do Domínio:** A camada de serviço deve ser completamente agnóstica em relação a convenções de rede (HTTP) ou detalhes intrínsecos de banco de dados, focando puramente na orquestração das entidades e regras do negócio.

4. Arquitetura do Front-end (React)

Objetivo Conceitual: Conceber uma Single Page Application (SPA) orientada a componentes que priorize a experiência do usuário (UX), a acessibilidade e o desacoplamento lógico.

O sistema deve possibilitar o envio de mensagens e o recebimento de respostas, a listagem do histórico de conversas, e o envio de documentos em anexo nos formatos txt e pdf.

A interface deve prover um ecossistema fluido para chat e upload de arquivos. O desenvolvimento client-side deve ser guiado pela clara segregação entre o que é exibido e como a aplicação se comporta:

- **Apresentação vs. Comportamento:** Os componentes visuais da interface (áreas de chat, listas, painéis de drag-and-drop) devem ser idealmente funções declarativas focadas apenas em renderizar dados.
- **Gestão de Estado Isolada:** Toda a mutação de estados, orquestração de chamadas à API e efeitos colaterais complexos devem ser obrigatoriamente extraídos para abstrações lógicas (Hooks customizados). Esta separação garante que a "inteligência" do front-end possa ser testada e mantida de forma totalmente independente da biblioteca de renderização visual.

5. Padrão de Entrega e Documentação

A rastreabilidade e a organização do código são fundamentais para sustentar arquiteturas desacopladas. Para a conclusão e submissão desta etapa, a equipe deve seguir estritamente o formato de entrega abaixo:

- **Separação de Repositórios:** A entrega oficial será feita através do envio de **dois links separados do GitHub** (um para o repositório do front-end e outro para o repositório do back-end). Essa divisão física de repositórios deve refletir a separação lógica das camadas do sistema.
- **Documentação Dedicada por Repositório:** Cada repositório deve ser autossuficiente e possuir sua própria base de documentos. É obrigatória a presença de um arquivo README.md contendo as instruções de execução e especificações técnicas de cada ambiente. Adicionalmente, sempre que ferramentas autônomas ou baseadas em IA gerarem artefatos de código, a presença de um arquivo AGENTS.md será exigida para documentar os prompts base e o escopo de atuação do agente no respectivo repositório.